

Week 3 Module Project A: Search and Adversarial Agents

Integrating pathfinding with minimax, alpha-beta pruning, and evaluation

CMPSC 480 - Student Question Deck

Assessment overview

Build a deterministic agent that plans paths and chooses actions against an opponent.

Coverage: Weeks 1-3: search formulation, A-star, minimax, alpha-beta, and evaluation

Points

20 total

Expected time

10-14 hours

Submission

Repository, tests, and engineering report

Learning objectives

- ▶ Reuse a stable search core for deterministic pathfinding.
- ▶ Implement depth-limited minimax and alpha-beta pruning.
- ▶ Design a state evaluation function with explicit feature semantics.
- ▶ Keep planning and adversarial reasoning behind separate interfaces.
- ▶ Measure correctness, pruning effectiveness, and decision quality.

Requirements checklist

- ▶ Use Python 3.11 and NumPy only unless the prompt explicitly permits another package.
- ▶ Preserve the published interfaces and make all tie-breaking deterministic.
- ▶ State assumptions, validate inputs, and handle empty, terminal, and unreachable cases.
- ▶ Include focused tests whose names explain the defect each test would expose.
- ▶ Report measured evidence; do not replace experimental results with unsupported claims.
- ▶ Do not mutate game states shared by sibling branches.
- ▶ Return a legal action for every nonterminal state with at least one legal move.

Task 1 - Architecture and pathfinding (4 points)

Define component boundaries and integrate BFS, DFS, and A-star pathfinding.

- a. Draw the boundary among game state, search problem, pathfinder, and adversarial agent. [1 points]
- b. Implement BFS, DFS, and A-star behind one pathfinding interface. [2 points]
- c. Demonstrate path validity and deterministic tie-breaking. [1 points]

Task 2 - A-star heuristic (3 points)

Design, justify, and test a domain-specific heuristic.

- a. Define the heuristic and its units. [1 points]
- b. Give an admissibility or consistency proof sketch. [1 points]
- c. Compare A-star with $h=0$ on at least three maps. [1 points]

Task 3 - Minimax decision engine (5 points)

Implement deterministic, depth-limited minimax with correct terminal semantics.

- a. Write minimax pseudocode with MAX, MIN, terminal, and cutoff cases. [2 points]
- b. Trace a depth-two tree with at least two actions per internal node. [2 points]
- c. Explain how depth is counted and how ties are broken. [1 points]

Task 4 - Alpha-beta and evaluation - part 1 (5 points)

Add pruning without changing minimax values and construct a defensible cutoff evaluation.

- a. State the alpha and beta invariants and the pruning condition. [2 points]
- b. Verify equality with plain minimax across generated small trees. [1 points]
- c. Define an evaluation function as a weighted feature sum. [1 points]

Task 4 - Alpha-beta and evaluation - part 2 (5 points)

Add pruning without changing minimax values and construct a defensible cutoff evaluation.

d. Measure the effect of ordering and evaluation features.

[1 points]

Task 5 - Integration and evidence (3 points)

Deliver an end-to-end agent and explain failures with reproducible evidence.

- a. Run an end-to-end scenario that uses both pathfinding and adversarial choice. [1 points]
- b. Test terminal roots, no legal actions, repeated states, and a narrow pruning window. [1 points]
- c. Report nodes expanded, prune count, decision value, and runtime. [1 points]

Edge cases and defensive programming

- ▶ The game is terminal at the root.
- ▶ A nonterminal state has no legal actions because of a domain defect.
- ▶ Different paths reach the same immutable state.
- ▶ All root actions have equal backed-up value.
- ▶ Alpha equals beta exactly after an update.

Submission instructions

- ▶ Submit the repository with modules, tests, and a reproducible demo command.
- ▶ Include a report with architecture, heuristic argument, evaluation features, and result tables.
- ▶ Provide a tagged final commit and exclude generated caches and credentials.

Do not submit credentials, generated caches, or unapproved libraries.

Grading rubric - part 1

Category	Points	Required evidence
Architecture and pathfinding	4	Architecture diagram, interfaces, and pathfinding tests.
A-star heuristic	3	Formula, proof sketch, and comparison against $h=0$.
Minimax decision engine	5	Minimax implementation, hand trace, and unit tests.

Grading rubric - part 2

Category	Points	Required evidence
Alpha-beta and evaluation	5	Alpha-beta implementation, evaluation ablation, and pruning metrics.
Integration and evidence	3	Integrated demo, adversarial tests, and concise report.

Reflection prompt

In 200-250 words:

Explain one integration defect that looked like an algorithm defect. Describe the boundary contract, the diagnostic evidence, and the final repair.